

(For example, humans watching graphs and reacting is not allowed.)

5. Specific observability (monitoring and alerting) MUST be in place, to pinpoint and attribute sources of load and growth.

6. Data ownership plans MUST include the entire lifecycle of the data, including:

1. Backup and restoration plans
2. Data retention policies that are tied to business goals and consider all potential legal implications (such as PII, personally identifying information)
3. Replication plans (Geo)
4. Cost analysis
5. Compatibility with existing data management features, like export and import

7. Data for any user-facing feature MUST reside, and be accessed through, a data store corresponding to the feature maturity.

1. For example, a General Availability launch REQUIRES data to be stored in, and accessed through, a production-quality Infrastructure-owned data store.

8. Changes MUST have documented rollout plans.

9. The points above MUST be reconsidered each time a feature experiences a lifecycle change (launch, significant growth, change in maturity or scope, sunseting) before the lifecycle change can take place. The responsibility to revisit belongs to feature owners, aided by Data Access experts.

10. Exceptions to any of the above MUST be thoroughly and permanently documented with risk assessment and business considerations, and approved by Senior Manager, Data Access or above, and the appropriate Product counterpart(s).

All Team Members

The following people are permanent members of teams that belong to the Data Access Sub-department:

Database Framework

The Database Framework

team develops solutions for scalability, application performance, data growth and developer enablement especially where it concerns interactions with the database.

```
{{< team-by-manager-slug manager="alexives" >}}
```

Database Operations

The Database Operations

team builds, runs, and owns the entire lifecycle of the PostgreSQL database engine for GitLab.com.

```
{{< team-by-manager-slug manager="rmar1" >}}
```

Durability

The Durability

team is dedicated to safeguarding and securing customer data that is stored by the GitLab application and set guidelines for data access. We strive to build

and maintain resilient infrastructure and improve the management of Redis, Sidekiq, and Gitaly.

```
{{< team-by-manager-slug manager="jarv" >}}
```

Gitaly

The Gitaly

team builds and maintains systems to ensure Git data of GitLab instances, and GitLab.com in particular, is reliable, secure and fast.

```
{{< team-by-manager-slug manager="jcaigitlab" >}}
```

Git

The Git team

develops Git in accordance with the goals of the community and GitLab, and integrate it into our products.

```
{{< team-by-departments "Git Team" >}}
```

SECTION: engineering/infrastructure-platforms/data-access/help.md

This guide is for Reliability and Support engineers to quickly and easily find the help they need on database-related emergencies.

```
{{% alert title="Note" color="primary" %}}
```

This page is intended for helping find resources during emergencies. If this isn't an emergency (IE: S1/S2 issue), instead ensure there are appropriately labeled issues so they can be triaged by the team.

```
{{% /alert %}}
```

Non-Emergencies

1. Label your issue to be triaged by the right team:

- For operational or configuration issues, label them ~"team::Database Reliability"
- For issues related to the packaged Postgres in Omnibus/Charts, label them ~"group::distribution"
- For application issues, label them for the team responsible for that feature.
- If you're not sure, take a look at the guide below for help identifying the right team.

1. If the issue is blocking or you need to escalate:

- For application issues, Post a detailed message in the channel for the team that is responsible for the related feature.
- For operational or configuration issues, post in ginfradatabasereliability (internal)
- For issues related to the packaged Postgres in Omnibus/Charts, post in gdistribution

(internal)

1. Start

If the emergency is related to configuration or operations like:

- Emergencies related to operating Postgres
- Connection errors
- Replication errors
- SSL Errors

Proceed to step 2. Configuration Errors

If not, proceed to step 3. Application Errors

2. Configuration or Operational Errors

If the emergency is related to an ongoing incident on gitlab.com or Dedicated customer, Follow the DBRE Escalation Process

If the emergency is related to a self-managed customer, reach out to the Distribution Team in gdistribution (internal) who manages self-managed configuration.

3. Application Errors

If the emergency is related to a single query, page, or endpoint, for example:

- A page (or class of pages) has a 500 and Sentry identifies it as a PG Timeout Error
- A long running transaction is identified as coming from a sidekiq worker
- A single query is identified as taking 10s on production and slowing down the site
- A migration is failing or has resulted in an emergency

Go to step 4. Single Source Issues

If not, go to 5. Widespread Issues

4. Single Source Issues

If the emergency is from an error that includes a feature category, go to 8. Reach out to a team based on feature category. As subject matter experts for a given feature, backend engineers are typically familiar with the involved database patterns and are typically best suited to solve issues related to their features even when the issue is related to database actions.

If the emergency is related to a migration, see 6. Determine Migration Source

If the emergency is related to a Rails controller, Sidekiq worker, API endpoint, or background migration, determine the feature category using details in our feature categorization guide, then go to 8. Reach out to a team based on feature category

If you need assistance to identify the source, go to 9. Escalating assistance

5. Widespread Issues

If the application (or component a major component ex: Sidekiq) is down or unresponsive due to what you believe to be a database related incident, that's an "All hands on deck".

1. Activate Development On-Call. While it may seem unnecessary, many backend developers are familiar enough with the application and database that they should be able to help isolate a source while trying to get database experts involved.

1. Reach out in the database, gdatabase, and ginfradatabasereliability channels (internal) for expert help using the @db-team (database capability) or @dbre (database reliability) group handles.

6. Determine Migration Source

The easiest way is using git, from the gitlab repository, run:

```
sh
git log --first-parent {path/to/migration.rb}
```

That should give you an output that includes a link to the merge request where the migration was added.

If that doesn't give a clear answer, you can look at the tables involved in the migration and take a guess at the team. See 7. Determine Source based on a Table

If it's still unclear what team to contact, go to 9. Escalating assistance

7. Determine a source based on a Table Name

Each database table has a documentation file that can be used to determine a corresponding group.

1. Look for the corresponding file named {tablename}.yml in <https://gitlab.com/gitlab-org/gitlab/-/tree/master/db/docs>

1. In the file, find the list of related featurecategories

1. Using the feature category, go to 8. Reach out to a team based on feature category

1. If there is more than one category, pick one from the list and start with that team

1. If it's still unclear what team to contact, go to 9. Escalating assistance. Be sure to include details about the table that's causing the issue, and why you believe it's involved.

8. Reach out to a team based on feature category

Even if the emergency is related to the database or has database words in it, the best first step is to contact the team responsible for that area of the application. The easiest way to figure out what team is responsible for that area is by feature category.

1. Using the feature category check the corresponding group in the Category-Team mappings

1. Reach out in that team's slack channel, and @mention the team's manager for assistance
1. If the team doesn't respond, go to 9. Escalating assistance.

9. Escalating assistance

When escalating an emergency, be as specific as possible and provide as many details as possible. Per communication guidelines, avoid acronyms whenever possible.

Always include:

- A link to the issue, Sentry error, incident, or Zendesk ticket
- The text of any error messages
- Links to any applicable charts
- Details about the query, migration, or issue

For Ongoing GitLab.com or Dedicated Incidents

1. Activate Development On-Call

1. If development on-call needs additional database expertise, reach out in database
1. If there's no response within 15 minutes, or the request is urgent, tag @db-team (Application) or @dbre (Infrastructure/Operations) in a thread on the original message
1. If there's no response to the ping within 15 minutes, and the request is urgent, use slack to find the phone number of the Database or DBRE manager and text or call them.

For Support Escalations

1. File a request for help issue
1. Reach out in database, include a link to the request for help

SECTION: engineering/infrastructure-platforms/data-access/database-framework/_index.md

Vision

Developing solutions for scalability, application performance, data growth and developer enablement especially where it concerns interactions with the database.

Mission

Focusing on the database, our mission is to provide solutions that allow us to scale to our customer's demands. To provide tooling to proactively identify performance bottlenecks to inform developers early in the development lifecycle. To increase the number of database maintainers and provide database best practices to the community contributors and development teams within GitLab.

Team Members

The following people are permanent members of the Database Team:

{{< team-by-departments "Database BE Team" >}}

Stable Counterparts

The following members of other functional teams are our stable counterparts:

Name	Role
-----	-----
Sampath Ranasinghe	Senior Product Manager, Geo and Database
Ben Prescott	Staff Support Engineer

Stable Counterparts to other teams

The Database Group is often called upon to provide consulting to other groups. To more efficiently support these requests we have created this stable counterparts table.

Meetings

Whenever possible, we prefer to communicate asynchronously using issues, merge requests, and Slack. However, face-to-face meetings are useful to establish personal connection and to address items that would be more efficiently discussed synchronously such as blockers.

- Database Group Sync every Tuesday and Thursday at 1:00 PM UTC
 - Tuesdays - we start with any ~infradev issues requiring reviews, then we focus on weekly priorities.
 - Thursdays - are optional and open agenda. Anyone can bring topics to the team to discuss. Typically we reserve the first Thursday after the milestone closes to hold a synchronous retrospective.
- Database Office Hours
(internal link); YouTube recordings
 - Wednesdays, 3:30pm UTC (bi-weekly)
 - (APAC) Thursdays, 3:30am UTC (bi-weekly, alternating)

Work

We follow the GitLab engineering workflow guidelines. To bring an issue to our attention please create an issue in the relevant project. Add the ~"group::database" label along with any other relevant labels. If it is an urgent issue, please reach out to the Product Manager or Engineering Manager listed in the Stable Counterparts section above.

What we do

The team is responsible for the PostgreSQL application interactions to enable high performance queries while offering features to support scalability and strengthen availability. PostgreSQL is the heart of Rails application, and there is no shortage of work to make GitLab more performant, scalable, and highly available from database perspective. Some of the current priorities

include implementing partitioning to improve query performance and creating tooling to enable development teams to implement their own partitioning strategies more easily. We are working on tools that will help developers "shift left" in their migration testing prior to deployment. We are always looking for ways to continuously care for the performance of our database and improve our developer documentation. For more in-depth details of what we are working on please review our Roadmap section below.

In order to follow what the database group is currently working on, we recommend watching our group's kickoff presentations for new milestones and the respective milestone planning issues.

Activity Log

Since end of 2021, we maintain an activity log to keep track of past projects and outcomes.

Planning

We use a planning issue to discuss priorities and commitments for the milestone. This happens largely asynchronously, but when we do need to discuss synchronously we discuss during the Tuesday team meeting timeslot.

Issue Weights

The database group is experimenting with using expected merge request count as an issue weight. Before each milestone starts, we'll ping each assigned issue without a weight and ask folks to add weights to them.

We decided to use merge request count as an issue weight for a few reasons:

- The process encourages folks to consider ahead how an issue could be broken down more and enumerate it in advance
- It's easy to describe and learn, making it easier for the team to come to a shared understanding
- Merge request rate is one of the main ways our team is measured

Process for weighting Issues

1. With an emphasis towards smaller more iterative changes rather than large changes that may take longer to review and merge, consider how many merge requests could this be broken into.

1. Add a comment enumerating the expected merge requests. For example:

- > Just one merge request to documentation
- >
- > One to gitlab for database changes, one for new functionality, one for documentation changes, and one to omnibus

1. Add the count as a weight. For example, if you think there could be one to gitlab for database changes, one for new functionality, one for documentation changes, and one to omnibus - you would assign /weight 4

Timeline for implementation

15.4 - 15.7: We'll ping each issue in the milestone without a weight and ask folks to add one to collect data

15.8 +: TBD

Triage rotation

We have a fairly simple triage rotation. Each week one team member is dedicated to triaging incoming issues for the database group. This allows for the rest of the team to focus on their current priorities with fewer interruptions. Each week, a bot will file an issue that gets automatically assigned to next team member in the rotation. We order the triage rotation by alpha-order based on first name to keep it very simple. If a team member is on PTO the week they are assigned, the issue will be re-assigned to the next person.

Issues needing triage can come in through many different paths. Some common areas to monitor while on triage:

- Newer issues (< 7 days old) with the ~database label that are not assigned to a group. Example search
- Newer issues that were assigned ~group::database but do not have a throughput label or ~database::triage labels. Example search
- Newer issues that were assigned ~database::triage and have not previously been reviewed
- When we get pinged on the gdatabase slack channel for assistance

When the triage team member discovers an issue requiring team attention some of the possible outcomes are:

- Directly address the issue if it is a simple fix
- Direct to our customer support counterparts as appropriate
- Add the ~database::triage label and review during team sync meeting
- Add a milestone and ping the manager, or label the issue ~workflow::scheduling
- Close as duplicate and link to the duplicate issue

The goal is to keep the number of issues for triage low and manageable.

Tip: In order to remove closed issues from the triage board, use this search and edit multiple issues at once to remove the ~database::triage label.

Boards

Database by Milestone

The Milestone board gives us a "big picture" view of issues planned in each

milestone.

Database: Build · Boards · GitLab.org · GitLab The build board gives you an overview of the current state of work for group::database. These issues have already gone through validation and are on the Product Development Build Track. Issues are added to this board by adding the current active milestone and group::database labels. Issues in the workflow::ready for development column are ordered in priority order (top down). Team members use this column to select the next item to work on.

Database: Validation

The validation board is a queue for incoming issues for the Product Manager to review. A common scenario for the Database Team validation board is when an issue is created that requires further definition before it can be prioritized. The issue typically states a big picture idea but is not yet detailed enough to take action. The Database Team will then go through a refinement process to break down the issue into actionable steps, create exit criteria and prioritize against ongoing efforts. If an issue becomes too large, it will be promoted to an epic and small sub-issues will be created.

Database: Triage

The triage board is for incoming issues that require further investigation for team assignment, prioritization, previously existing issues, etc. Within the Database Group we have implemented a weekly triage rotation where one team member is responsible for monitoring this board for timely responses.

Say/Do Ratio

We use the ~Deliverable label to track our Say/Do ratio. At the beginning of each milestone, during a Database Group Weekly meeting, we review the issues and determine those issues we are confident we can deliver within the milestone. The issue will be marked with the ~Deliverable label. At the end of the milestone the successfully completed issues with the ~Deliverable label are tracked in two places. We have a dashboard in Tableau that will calculate how many were delivered within the milestone and account for issues that were moved. Additionally, our milestone retro issue lists all of the ~Deliverable issues shipped along with those that missed the milestone.

Roadmap

The Database Group

Roadmap

gives a view of what is currently in flight as well as projects that have been prioritized for the next 3+ months.

Weekly Team Updates

The enablement section is using status issues in order to provide regular status updates. Each week, the team's engineering manager posts general announcements,

and members of the team post updates on their in progress projects.

These issues can be found here (internal).

Documentation

We document our insights, road maps and other relevant material in this section.

- 1. Database Lexicon - terms and definitions relating to our Database
- 1. Database Strategy: Guidance for proposed database changes
- 1. On table partitioning (February 2020)
- 1. Postgres: Sharding with foreign data wrappers and partitioning
- 1. Sharding GitLab by top-level namespace
- 1. Sharding with CitusDB (April 2020)
- 1. Table partitioning: Issue group search as an example (March 2020)
- 1. Working with the GitLab.com database for developers
- 1. Database schema proposals for Container Registry (September 2020)
- 1. Workload analysis for GitLab.com (October 2020)
- 1. Multi-database Background migrations (October 2021)

Performance Indicators (Internal)

- 1. Enablement::Database - Performance Indicators Dashboard
- 1. Average Query Apdex for GitLab.com
 - Master
 - Target: 100ms - Tolerable 250ms
 - Target: 50ms - Tolerable 100ms
 - Replicas
 - Target: 100ms - Tolerable 250ms
 - Target: 50ms - Tolerable 100ms

Common Links

- Slack Channel
gdatabase -
Official Business
- Slack Channel
db-lounge - Team Chat
- Database Epics
- Database Subgroup - Issues and
templates related to team processes.
- Product Development Timeline
- YouTube: Database Team Playlist
- YouTube: Database Office Hours Playlist

SECTION: engineering/infrastructure-platforms/data-access/database-framework/activity-log.md

Overview

This page captures the database group's activity and documents the outcomes, key results and some takeaways (most recent first). We've started doing this towards the end of 2021.

2022

March 2022

Starting in November 2021, we performed a migration file cleanup with these goals:

- Improve the performance of GitLab CI jobs.
- Remove maintenance cost for old migrations.
- Improve initialization speed for new GitLab instances.

We decided to remove all of the migration files before version 14. We implemented a script that squashed multiple migrations files into one file (initschema).

We drastically reduced the number of migration files. Using a migration squashing strategy, we deleted thousands of regular migrations, post-migrations, background migrations, and RSpec files. In the future, when building a new database from scratch, the system does not need to open and parse all migrations before version 14 because they are squashed into one file. This change contributed to a performance improvement.

After the cleanup deployment, we saw a performance optimization in our CI, which led to a significant time reduction of the rspec migration job.

2021

November 2021

This month, we shipped three changes to GitLab.com which led to a 20% reduction in total database size (about 3.3 TB total reduction):

1. Priority reindexing and reindexing support for very large btree and GIN indexes
1. Drop deprecated tables
1. Partitioning-based retention strategy for time-decay data

We implemented priority reindexing which allows us to target specific indexes as needed. This helped to reduce index bloat built up throughout a large data migration. It reduced GitLab.com index bloat by >500 GB total, which translates to reducing relative btree bloat across the site from 25% to 17% and freeing up 500 GB worth of disk space in the cluster. This has stabilized the longer-term index bloat forecast.

We released a data retention strategy for time-decay data which is based on dropping partitions rather than deleting records. This solves a major problem on GitLab.com, where a deletion-based strategy hasn't been able to keep up and generated load. After switching on the new strategy, we immediately dropped 2 TB worth of data in webhooklogs.

SECTION: engineering/infrastructure-platforms/data-access/database-framework/identifying-database-issues.md

This guide is intended for folks who need to determine the DRI team when examining a database issue.

Migrations

The easiest way is using git, from the GitLab repository, run:

```
sh
git log --first-parent {path/to/migration.rb}
```

The code path/to/migration.rb can be found in the backtrace when the migration fails. Migration code files

start with a date-time stamp and are in db/migrate/ or db/post-migrate/. Or, if you can find the date-time stamp, for example 20240113071052, anywhere in the log output from the customer, that will uniquely match a migration filename in one of these locations.

That should give you an output that includes a link to the merge request where the migration was added.

If that doesn't give a clear answer, you can look at the tables involved in the migration and take a guess at the team. See Finding a team by table name.

Queries

It's a bit more complicated to determine query sources, because they come from a lot of places.

If the issue is related to a Rails controller, Sidekiq worker, API endpoint, or background migration, determine the feature category using details in our feature categorization guide, then use reach out to the team listed in the lookup to contact the team.

If you don't have a source that includes a feature category, you'll need to make a guess based on the table name in the query. You can follow Finding a team by table name.

Finding a team by table name

Each database table has a documentation file that can be used to determine a corresponding group.

1. Look for the corresponding file named {tablename}.yml in the database dictionary
1. In the file, find the list of related featurecategories
1. Using the feature category, reach out to the team listed in the lookup
1. If there is more than one category, pick one from the list and start with that team

Getting a team from a feature category

If you have a feature category, the best way to determine the team to contact is by using the feature category lookup.

SECTION: engineering/infrastructure-platforms/data-access/database-framework/postgresql-upgrade-cadence.md

PostgreSQL yearly upgrade cadence

Starting with GitLab 16.0, we follow a yearly upgrade cadence for PostgreSQL:

1. With every major GitLab version, we are increasing the minimum required PostgreSQL version to the next major version. Some examples:

- In GitLab 17.0, PostgreSQL 14 will become the minimum supported PostgreSQL version.
- In GitLab 18.0, PostgreSQL 16 will become the minimum supported PostgreSQL version.

1. We will be announcing the deprecation of the current minimum PostgreSQL version one year in advance, with the release of each major version of GitLab:

- In GitLab 15.0 we have announced the deprecation of PostgreSQL 12 (to be removed in GitLab 16.0).
- In GitLab 16.0, we will announce the deprecation of PostgreSQL 13 (to be removed in GitLab 17.0).

1. GitLab.com will be upgrading to the next major version every year during Q2. Our infrastructure will be preparing the upgrade in Q1 of every year, and execute the upgrade during Q2.

1. Optional support for the next major PG version in Omnibus and charts will be tested and validated with the release of each major version of GitLab. The next major PostgreSQL version will become available between .0 major version and .4 minor version, and then will become the default for fresh installations in the middle of the release cycle (~ minor version .6). For example:

- In GitLab 16.0, we will test and validate PostgreSQL 14 support in Omnibus and charts.
- Between GitLab 16.0 and 16.4, we will add PostgreSQL 14 to Omnibus and charts as an optional supported version.
- Around 16.6, PostgreSQL 14 will become the default for Omnibus fresh installations.
- This change is not required for GitLab 15.x, as we are keeping PostgreSQL 12 as the minimum, and support for PostgreSQL 13 has been already added in GitLab 15.0.

At a glance:

| GitLab Version | Minimum Supported Version | Optionally Supported Version | Omnibus Default
(Fresh installs) | Release notes | GitLab.com |

|-----|-----|-----|-----|
-----|-----|-----|

16.6	PG13	PG14	PG14
GitLab.com will upgrade to PG16 in FY25-Q2			
17.0	PG14	PG16	PG14
Support for PG16 will be added to omnibus and charts between 17.0 and 17.4 PG14 will be removed in 18.0			
17.6	PG14	PG16	PG16
18.0	PG16	PG17	PG16
GitLab.com will upgrade to PG17 in FY26-Q2			

PostgreSQL Minor Release Cadence

Each major Postgres version receives bug fixes and, if need be, security fixes that are released at least once every three months in what Postgres calls a "minor release." Many people will be more familiar with the term `patch`, which more or less refers to the same thing.

Postgres usually releases minor releases every 3 months. The target date for these releases are, unless otherwise stated, the second Thursday of February, May, August, and November. If the Postgres release team determines that a critical bug or security fix is too important to wait until the regularly scheduled minor release, they may make a release available outside of the minor release roadmap.

We can view the Postgres minor release roadmap here:
<https://www.postgresql.org/developer/roadmap/>

The Database Operations Team (DBO Team) may or may not choose to apply each minor release after considering its release notes. When considering whether to install a minor release, we evaluate whether the bug fixes and security enhancements included are likely to be relevant in Gitlabs environment. We also take a look at community forums across the web to note if anyone reports issues with installing that minor release.

Individual minor releases should be evaluated by different people from the DBO team, round robin style. It should not be one team member's sole responsibility to be aware of upcoming postgres minor releases that we may want to install. The team member responsible for evaluating the next minor release is assigned when the current minor release issue is closed.

New Minor releases should be evaluated by using the postgresql minor release template to open an issue in the DBO Issue Tracker Repo.

New Minor releases should be installed after opening a change request in the production repo

The DBO team will usually default to waiting at least a week after a minor release is made available before beginning evaluation.

It is very important that minor releases are first installed in the staging environment, and left to run there for at least 2 weeks before they are installed in the production environment.

SECTION: engineering/infrastructure-platforms/data-access/database-framework/stable.md

Overview

A central tenet of the Core Platform department is to enable other teams to be more self-sufficient in areas such as, in this case, the database. The Database Group is here to provide a helping hand in areas such as database design, query efficiency, database tooling and more. In order to be more efficient and to avoid diffusion of responsibility when providing database expertise to other groups, we are providing a list of database stable counterparts and which area of expertise is owned by each team member.

Stable Counterparts

If you have a need for database assistance please review the table below for the stable counterpart for your area. If one is not listed please feel free to reach out directly to the gdatabase slack channel.

Responsibilities of the stable counterpart

1. Following the topic, knowing the latest updates (which makes us more efficient since we don't all have to keep track of everything)
1. Actively work with the respective group, e.g. to implement a database design
1. Translate work from the topic into our group and involve other folks from our group to contribute as needed

Feature (alpha order)	Stable Counterpart	GitLab Handle
---	---	---
AI Tanuki Bot	Matt Kasa	@mattkasa
Background Migrations	Diogo Frazão	@dfrazao-gitlab
CI Partitioning	Simon Tomlinson	@stomlinson
Consumables Management (storage, compute, seats, usage, etc)	Kras Angelov	@krasio
Container Registry	Simon Tomlinson	@stomlinson
Database Testing Framework	Matt Kasa	@mattkasa
Database query interceptor	Jon Jenkins	@jonjenkins
Embeddings Database	Matt Kasa	@mattkasa
Primary Key Migrations	Kras Angelov	@krasio
Table Size Reduction	Simon Tomlinson	@stomlinson
Vulnerability Management	Simon Tomlinson	@stomlinson
Work Items	Matt Kasa	@mattkasa

SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/citus.md

Sharding GitLab with CitusDB

This is a working document to outline the decision making process with respect to using CitusDB as a database sharding solution for GitLab on GitLab.com.

Citus Community

We were exploring the Citus Community offering as part of our efforts to explore CitusDB as a

sharding solution. Citus Community is licensed under the GNU Affero General Public License v3.0 (GNU AGPLv3). GNU AGPLv3 is listed in our handbook as an unacceptable license requiring legal approval for use.

On April 15, 2020 we sought advice from our legal counsel. There were enough questions and concerns about the GNU AGPLv3 license that we decided to discontinue our usage research into Citus Community. Notes and agenda can be found [here](#) (only accessible to GitLab team-members).

Citus Enterprise

April 20, 2020 - Due to licensing costs we have decided not to pursue Citus Enterprise for a sharding solution. We will focus our efforts on PostgreSQL Partitioning with foreign data wrappers (FDW).

SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/fdw-sharding.md

PostgreSQL 11 sharding with foreign data wrappers and partitioning

This document captures our exploratory testing around using foreign data wrappers in combination with partitioning. The idea is to implement partitions as foreign tables and have other PostgreSQL clusters act as shards and hold a subset of the data.

Background

With PostgreSQL 11 declarative partitioning, we can slice tables horizontally. That is, we keep working with the top-level table as usual but underneath we organize the data in multiple partitions. Those can be thought of as regular tables that are being attached to the top-level table (much like table inheritance in PostgreSQL).

In order to implement partitioning, we choose a partitioning key and a partitioning method (hash or range based - read more about it in the docs). With any of the methods, a partition contains data only for a subset of the partitioning key's values. Queries benefit from this only if they contain a filter on the partitioning key. In that case, the query planner statically derives which partitions have to be scanned for a particular query. Best case it only scans one partition to satisfy the query and excludes all other partitions.

Standard PostgreSQL partitioning creates all partitions equal and on the same physical cluster. Therefore, partitioning is not a built-in way to distribute data across multiple clusters. Instead, it implements logical data organization and splitting but does not natively support distributing data. This is where PostgreSQL foreign data wrappers come in and provide a way to access a foreign table just like we are accessing regular tables in the local database. In our exploratory scheme, each partition is a foreign table and physically lives in a separate database. We call this a "shard", which can also live in a totally separate database cluster.

The PostgreSQL community has a roadmap to build sharding capabilities into native PostgreSQL in upcoming versions.

Initial setup

In this demo, we go through the process of setting up the schema.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/watch?v=MIZFtM84x44" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

In our example, we picked issues table as we did in previous examples. Choosing the projectid as the partitioning key (because that is already available), we re-create the top-level table with HASH(projectid) based partitioning:

```
sql
CREATE TABLE public.issues (
  id integer NOT NULL,
  title character varying
  -- ...
  projectid integer,
  -- ...
) PARTITION BY HASH (projectid);
```

As a next step, we prepare two shards to contain the actual partitioned data: Two local databases shard1, shard2 - which could also live on a separate PostgreSQL cluster.

```
sql
CREATE EXTENSION IF NOT EXISTS postgresfdw;
```

```
CREATE SERVER IF NOT EXISTS shard1 FOREIGN DATA WRAPPER postgresfdw OPTIONS (dbname 'shard1'
host 'localhost');
CREATE SERVER IF NOT EXISTS shard2 FOREIGN DATA WRAPPER postgresfdw OPTIONS (dbname 'shard2'
host 'localhost');
```

We then proceed and create partitions on the local database. Note that we are creating those as foreign tables and distribute them across shard1, shard2:

```
sql
CREATE FOREIGN TABLE parts.issues0 PARTITION OF public.issues FOR VALUES WITH (modulus 8,
remainder 0) SERVER shard1;
CREATE FOREIGN TABLE parts.issues1 PARTITION OF public.issues FOR VALUES WITH (modulus 8,
remainder 1) SERVER shard2;
-- ...
```

Now the local database is prepared but we still have to create the actual tables in the shards. For each of the partitions, we create a regular table along with indexes in the shard1, shard2 databases.

A limitation at this point is that we cannot add foreign keys on these tables to any table that does not reside in the same shard.

With this setup, we can now run queries like the following and see them accessing one partition only which lives on the remote shard:

```
sql
SELECT
FROM issues
WHERE projectid = ?
```

We can continue to work with the issues table as usual. If we added more immutable conditions to the query, those would be pushed down on the shard.

Schema migrations

In order to change the existing schema, we've discussed two examples: Adding and dropping a column.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/watch?v=nt4Khi9Gr3o" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

The example migrations need coordination in a sense that the foreign table's schema needs to be a subset of the actual table. For adding a column, we would first add the column to all the actual tables on the shards and only later make this visible by adding the column to the foreign tables (the other way around for dropping columns).

Reference tables

So far, the shards only contain regular tables for partitioned data. That data cannot reference data that does not reside on the shard. For example, we might have a users table that isn't sharded and we want it to be available globally across shards.

In this step, we explore using logical replication to replicate selected tables across shards and make them available locally. This allows us to connect to a shard directly and work with the reference table and one of the partitions directly. However, this requires knowledge of which shard holds the partition of data we are actually interested in.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/watch?v=ztQtNmSYmEo" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

In this example, we setup logical replication slots for both shards and publish two tables: user, projects. On the shards, we subscribe to the publication and start replicating the reference tables locally.

This will allow us to run queries like the following:

```
sql
SELECT
FROM users
JOIN issues0 as issues -- note this is a single partition!
WHERE users.id = ? AND issues.title ~ ?
```

This is also possible on the main database of course. We've been exploring this as an option to run more queries directly on the shard. It remains to be seen how useful this is really.

Summary

This has been an exploration of combining foreign data wrappers with partitioning. PostgreSQL does not natively support sharding and distributing data across multiple physical clusters (yet). Foreign data wrappers serve as a tool to read data from remote servers and can be used to distribute data.

However, this setup comes with a lot of complexity and limitations compared to a classic PostgreSQL setup:

1. Need to maintain connection details to foreign servers.
2. Difficult to maintain consistent schema across a set of clusters.
3. Foreign keys are only supported locally.
4. Overhead risk when partitioning key is not available is higher when data is remote.
5. Risk of certain suboptimal joins and filters if not being pushed down to foreign table.
6. Performance degradation if multiple shards need to be accessed for a query.
7. Balancing increase in query planning times with execution times.
8. Each shard requires its own HA cluster.
9. No parallel scans of foreign tables.
10. Updates to reference tables still need to go to the main cluster creating a bottleneck for write-scalability and a single-point of failure.
11. No global transaction management.

In order to really benefit from this approach, we'd have to shard relevant tables by the same dimension and make it possible to execute queries directly on shards.

With a lot of different access patterns in GitLab, this seems not feasible before we agree on an application-wide sharding key and deal with conflicting access patterns (e.g. by means of service extraction or isolation).

Fortunately, partitioning is the groundwork that allows us to tackle the same set of problems without introducing much of the complexity coming from distributing the data physically. As such, introducing partitioning throughout the application is an iterative way of working towards a more scalable database solution in general.

We are going to tackle the audit log database design as a first step to introduce the groundwork for partitioning and prepare the application to make wider use of this pattern.

Author: Andreas Brandl

SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/gitlab-com-database.md

A developer's guide to working with the GitLab.com database

GitLab.com is powered by a large PostgreSQL database ("the database" in this doc) which is often used as a point of reference in terms of scale - after all, this is the largest installation of GitLab we have access to.

From a development perspective, it is often necessary to gather statistics and other insights from this database - for example to provide insights for query optimization during database review or when we need more insight into data distribution to inform a product or development decision.

This overview is aimed at developers wanting to access the database and also explains how to get this type of access in the first place.

Types of access

There are a couple ways to directly access the database for developers:

Direct access to production

Direct database access to a production replica

This is the most straight forward type of access: You'll have access to a psql console to an actual replica of the database (which typically serves live traffic). The console will be read-only and a 15s statement timeout must be adhered to.

How to get access: File an access-request and ask for production database console access (role: db-console) to receive permanent SSH access, or use Teleport to request temporary access (Recommended). See below for setting up SSH access.

Access to the "archive" replica

In this case, you'll have access to a psql console to the "archive" replica. This is a replica of the database which does not serve live traffic and is prepared to be used in a rather "exploratory" fashion (suitable for rather analytical queries, too). The console access is read-only and a 15 minute statement timeout applies. You can verify this by going db console and issuing `show statementtimeout` command.

How to get access: File an access-request and ask for console access to the archive database in production (role: db-console-archive). See below for setting up SSH access.

Rails console access

A Rails console can be used to access the database, too. Use with extreme care as this is typically a read-write console.

How to get access: File an access-request and ask for Rails console access in production (role: rails-console)
to receive permanent SSH access, or use Teleport to request temporary access (Recommended).
See below for setting up SSH access.

Setup SSH configuration for direct access

In order to setup direct access, you will have to configure SSH to use the bastion instances as a proxy:

1. Setup bastions to access production (gprd)
2. Setup bastions to access staging (gstg)

This is an example configuration for a user named joe (replace this with your gitlab email user) to access psql and Rails consoles in gprd:

text

```
GCP production bastion host
Host lb-bastion.gprd.gitlab.com
    User      joe
```

```
Host gprd-rails
    User      joe-rails
    StrictHostKeyChecking no
    HostName  console-01-sv-gprd.c.gitlab-production.internal
    ProxyCommand ssh lb-bastion.gprd.gitlab.com -W %h:%p
```

```
Host gprd-psql
    User      joe-db
    StrictHostKeyChecking no
    HostName  console-01-sv-gprd.c.gitlab-production.internal
    ProxyCommand ssh lb-bastion.gprd.gitlab.com -W %h:%p
```

```
Host gprd-psql-archive
    User      joe-db-archive
    StrictHostKeyChecking no
    HostName  console-01-sv-gprd.c.gitlab-production.internal
    ProxyCommand ssh lb-bastion.gprd.gitlab.com -W %h:%p
```

With this setup, the user joe could use the following commands:

```
sh
ssh gprd-rails      opens a rails console
ssh gprd-psql       opens a psql console
ssh gprd-psql-archive opens a psql console on a replica
```

DatabaseLabs

Use postgres.ai to work with a thin clone of the database with psql

We have access to postgres.ai which gives us the ability to work with a thin clone of the database. This has the benefit of providing a fully isolated read-write database for a single user. Thin clones are inexpensive to create and can be used and destroyed as needed. This includes direct psql access to the thin clone.

With the extended setup, you'll be able to use a psql console on a thin clone, too. This is the most flexible way of working with the database, as this even allows you to e.g. create new tables (e.g. for intermediate results) as needed.

Query plans and optimization

In order to evaluate query performance, we have the following tools available:

1. Retrieve query plans through ChatOps on Slack
1. Use database-lab Slack channel to work with a thin clone of the database
1. Use postgres.ai (currently under evaluation) to fully immerse into working with a thin clone to optimize queries

Where do I get help with setting up access or any other questions?

Please reach out to database on Slack and consider adding to this page afterwards.

Resources and Tips

Process

Various Product and Engineering processes are available to advocate database improvements that may not fit anywhere in the feature development roadmaps.

1. Leverage the Planning as a team

According to this finding, more than 50% of known query performance issues were identified long time ago. Leverage the planning as a team to prioritize these issues with Product. The process is being implemented by Introducing Error Budgets to Stage Groups.

1. Follow the instructions in Prioritize technical decisions when collaborating with Product in release planning, which also mirrored in the Product handbook page.
1. Consider Engineering Allocation for tech-debt issues, which is also mirrored in the Product handbook.
1. Keep the Architecture process in mind when running into large scale fundamental design challenges.

Best practices

Common issues seen in the application codebase are N+1 queries, inappropriate use of CTEs, and read-only queries not leveraging read replicas.

1. Guidance on N+1 Queries
 1. Query Recorder
 1. Performance guidelines
 1. Merge request performance guidelines - Query counts
 1. Merge request performance guidelines - Cached queries
1. Use CTEs wisely
1. Use read replicas when possible

Knowledge sharing

1. Video recording - Techniques for researching inefficient DB queries. Highly recommended!
1. Using Elasticsearch with PostgreSQL slow logs for troubleshooting.
1. Slow queries chart to identify potential sub-optimal queries.

Tools

1. Postgres.ai for testing with production like data.
1. Query plans and optimization.

SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/lexicon.md

Commonly Used Terminology

This is not a comprehensive list of all of the commonly used terms, but rather it is a list of terms that are commonly confused or conflated with other terms. In each section we will identify common phrases, define our specific usage and list external references for the term in question.

Partitioning

For our purposes, when referring to partitioning we are referring to a local optimization of splitting one large table into many small tables. Additionally, this is done in a way such that the application accessing this partitioned table is mostly unaware of the changes. Queries benefit from partitioning only if they use the partitioning key as a filter.

Definition - "Partitioning refers to splitting what is logically one large table into smaller physical pieces." Reference: PostgreSQL: Documentation: 11: 5.10. Table Partitioning

To learn more about the implementation details and performance improvements of partitioning please read the Database Team handbook entry: Database Partitioning

Sharding

The simplest definition of sharding is the process of breaking up portions of your database (e.g. large tables) into smaller tables and distributing those smaller tables across multiple servers. Where sharding can get confusing, especially with PostgreSQL, is that most definitions of sharding use the term partitioning to describe how the tables are broken down and distributed. This can lead to the terms sharding and partitioning being conflated.

For our purposes, when referring to sharding we are referring to horizontally distributing data across multiple database clusters ("shards"). Each shard only contains a subset of the overall data.

Application Sharding

Another variant of sharding is application sharding. This typically refers to an architecture that implements routing logic to route data requests from the application to the correct data repository. During our discussions in the Sharding Working Group we typically referred to the Shopify Pods Architecture as an example of application sharding.

Foreign Data Wrapper (FDW)

Foreign Data Wrappers allow for accessing remote data from multiple data sources. FDW can be used to create an integrated view from your PostgreSQL instance into many different types of resources, but for our purposes we are focusing on integrating multiple PostgreSQL servers. Foreign Data Wrappers (FDW) are an implementation detail that can change your PostgreSQL partitioning implementation to what is considered a Built-in Sharding implementation.

Partitioning with FDW = Sharding

With PostgreSQL, partitioning is the first implementation step taken on the path to sharding using Foreign Data Wrappers (FDW). More information and implementation details can be found in the Database Team Handbook Entry: PostgreSQL 11 sharding with foreign data wrappers and partitioning

SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/partitioning.md

Introducing PostgreSQL table partitioning to GitLab's database

This is a working document to discuss how we are going to introduce table partitioning to GitLab.

Motivation

The PostgreSQL database we run for GitLab.com has grown to over 5 TB in total size as of early 2020. However, the total database size is not the driver to introduce partitioning but the size of individual tables is:

We can see here that there are individual tables larger than 100 GB, some even go up in the terabytes range.

There is one notable table that we can ignore in this discussion: `mergerequestdifffiles` which is the largest table and makes up more than 30% of the total database size today. This is being addressed by externalizing this data and move it into cloud storage.

The other tables shown here is where GitLab's most used features have their data:

CI: Build jobs, trace sections, artifacts and pipelines

Merge requests: Descriptions, commits and notes

Issues: Descriptions and notes

Web hooks

Events, audit events and notifications

Growing table sizes

Table size has a direct impact on query performance and execution times. Up until this point, we've been improving query times by adding the appropriate indexes only. This particularly improves queries with low selectivity (very targeted ones).

The impact of growing table sizes is:

1. Index maintenance overhead grows
2. Index lookup performance becomes worse
3. Data is physically spread out over a large table leading to unnecessarily high IO
4. Managing table bloat becomes more difficult
5. Relevant working set does not fit into memory anymore, leading to an increase in IO

Example: Issue group search

The gitlab-org group has about 135k issues out of a total of 23M issues on GitLab.com.

We can perform a text search on the group's issues and look for all issues with "foobar" in their title or description along with other filters we can apply: It takes about 8s to just perform the text search in this group.

Under the hoods, we rely on trigram indexes for text search. Simplified, there are two ways to do this:

1. Text search first: Use the global trigram index to find all issues matching "foobar" and then apply other filters
2. Other filters first: Apply the group filter and permission checks first to find issues in the group and perform an online text search on their text data.

Which of these approaches is more efficient depends on data distribution and each of the approaches becomes slow the more data we have.

Problem 1: Large trigram index

In approach 1 we use the global trigram index first. Finding relevant records with "foobar" already takes 2.8s today, even though the query is very low selectivity (it yields <2,000 records) and rather well suited for the trigram index.

The trigram index is global: There is only one GIN index available (per column). This index structure is already 19 GB in size as of today (for issues.description).

Problem 2: Many issues per group

In approach 2, we first find all issues in the group we have access to. This uses btree indexes as usual but overall this yields 135k issues. Now we can't use the trigram index anymore to perform the text search but instead have to read all text data into memory and perform a sequential online search. The more issues a group has (and the larger the text size), the longer this approach takes.

Table partitioning to the rescue

In this example, table partitioning breaks down the issues table into manageable chunks aka partitions: Let's break the table down into 128 partitions using hash partitioning on the top-level namespace. Each partition contains issue records for roughly 1/128 of the total issues. Each top-level namespace, in our example that's gitlab-org, has their issue data in exactly one partition.

How does this change the situation?

1. Each partition has their own trigram index and those are now much smaller (containing only roughly 1/128 of records).
2. The scope of our search example is per group: The query planner uses this information to directly navigate to the one relevant partition for our top-level namespace and thereby excludes scanning all others.

With the much smaller trigram index, we can perform the text search in much less time. An additional improvement for this particular problem is to use a combined GIN index.

We analyzed the impact of partitioning for group-based issue search (summary, issue).

The GitLab database schema: Partitioning keys

This is a simplified view of a part of GitLab's database schema: Largest tables on GitLab.com along with a few of their references. The color indicates how large the table roughly is: Deep red (> 200 GB), orange (> 100 GB), yellow (> 60 GB), blue (else). For detailed sizes please refer to the image at the top of the doc.

(Disclaimer: Highly simplified, WIP and likely not detailed enough)

Unsurprisingly, most of the tables relate to a Project: User-specific data aside, a Project is referenced by most of the large tables shown here (directly or indirectly).

Moreover, a project is always part of the namespace hierarchy and as such there is always exactly one top-level namespace for a project.

As such, we can consider a project or a namespace as a partitioning key. While most of GitLab's features happen inside a project, there are types of searches that are scoped on a namespace. In our example above, the top-level namespace is the search scope. Hence, the top-level namespace is likely a good partitioning key in general.

Partitioning by top-level namespace

In order to partition a table by their top-level namespace, the table has to contain a reference to the top-level namespace (a `toplevelnamespaceid` column). None of the existing tables have this for a good reason: The schema is normalized and the top-level namespace can be figured out through the hierarchy in namespaces.

Adding the top-level namespace reference to a table, e.g. `issues` is a form of denormalization. It is necessary to do this in order to partition the table by top-level namespace.

The downside of denormalization is that there may be situations where a mass-update is necessary. In our case, this happens when we move a project to a different top-level namespace. In this case, all the records with a reference to the top-level namespace for the project have to be updated. Without denormalization, we would just update a single reference.

Hash partitioning strategy

We analyzed how issues distribute across partitions using a hash-based partitioning strategy (by top-level namespace). This is how the number of issues distributes across a set of 128 partitions (find more graphs for 64, 256 and 512 partitions). The red line indicates the desired optimal distribution (1/128 per partition).

Implementation roadmap

This section discusses several topics we already identified that have to be tackled along the way.

Support for advanced PostgreSQL features

The way we track the database schema needs to support advanced PostgreSQL features in order to implement partitioning. The currently used `schema.rb` does not support syntax for partitions, triggers and other advanced features we're likely going to need.

This means we're going to replace `schema.rb` with `structure.sql` and transition over to managing the database schema with SQL instead of the Rails abstraction for it.

Issue: Use `structure.sql` instead of `schema.rb`

Handling foreign keys to partitioned tables

PostgreSQL 11 does not support foreign keys referencing partitioned tables. We need to find ways to remove foreign keys from a table and still retain what we use foreign keys for:

1. Referential integrity throughout the database
2. Cascading deletes

In order to support cascading deletes, a trigger-based solution seems usable and gives us the same functionality. We need to implement handling the trigger definitions nicely.

Without foreign keys, we need to make the application more robust to flaws in referential integrity. Plus, we should have ways of detecting and possibly also fixing inconsistent data.

Issues:

Referential integrity without foreign keys

Cascading deletes without foreign keys

Strategy to migrate an existing table into a partitioned table

In order to migrate an existing, large table into a partitioned table we have to have an online migration strategy available. It has to migrate a lot of data and copy it over into a partitioned table hierarchy. This strategy has to be transparent to the application, i.e. we can't have a downtime from this. It should happen in the background with minimal impact on the production system.

Issue: Migration strategy for existing tables

Enhancing the schema to contain the partitioning key

Depending on the choice of partitioning key, we have to enhance the schema of relevant tables to contain the partitioning key. In our example above, we would add the reference to the top-level namespace to the issues table. As a consequence, we'll have to implement a background job to update this reference whenever a project is being transferred to another top-level namespace (which is deemed to be a rare event but it potentially affects a lot of records at once).

Enforcing the use of partitioning keys

In order to benefit from table partitioning, queries must be formed in a way that they always include the partitioning key. Oftentimes this is an artificial construct which doesn't change the semantics of the query. For example, a filter by `projectid` may semantically be enough and the additional filter on the partitioning key is redundant to that. However, it's important to include the partitioning key so that the query planner can exclude irrelevant partitions.

Often, we can derive the partition key from the URL already: Many queries that happen for requests inside the `gitlab-org` namespace likely should have the top-level namespace as a filter. We might consider using `activerecord-multi-tenant` (by CitusData), which has an automatic approach to detect the partitioning key and make sure queries contain the appropriate filter by partitioning key for relevant models.

Issue: Explore usage of `activerecord-multi-tenant` to enforce partitioning key in queries

Results:

1. Implementation example using the gem
2. The gem also takes care of setting the partitioning key when creating records
3. For mass-updates of the partitioning key, we might need to implement a background job (see above)
4. We may want to add a check to make sure we're always adding the partitioning key to queries. Here is an example with parsing SQL (it may be possible to hook this into ActiveRecord at a

higher level, too).

Type of partitioning and schema handling for partitions

There are different types of table partitioning available in PostgreSQL:

RANGE/LIST
HASH

For RANGE/LIST partitioning, partitions are created for an explicit range or list of values. With a growing dataset this means that an automatic way to create new partitions has to be implemented.

Hash-based partitioning instead allows us to create a fixed number of partitions to contain the full space. The drawback here is that refining the partitioning scheme to create more, smaller partitions is a bit more involved and splitting a partition into two smaller parts is possible but has to be implemented properly.

Hash-based partitioning is a static approach in context of partition creation and handling. As such, it is simpler to manage because we can simply create all partitions ahead of time and as part of the static database schema.

Author: Andreas Brandl

SECTION: engineering/infrastructure-platforms/data-access/database-framework/doc/strategy.md

Database Strategy: Guidance for proposed database changes

GitLab is offered as a Single Application with a Single data-store. This handbook entry is meant as guidance for when you encounter a situation where you are considering changes or additions to our data-store architecture. For information on tooling, migrations, debugging and best practices please read the Database guides section in GitLab Docs.

Requirement

When you propose any database additions, updates or deletions it is required that you have participated in a Database Review prior to deployment (best early in development).

PostgreSQL

The GitLab web app uses PostgreSQL for our persistent database information. We have dropped support for MySQL and will be requiring PostgreSQL 11 with the release of GitLab 13.0.

When to consider another data-store

Over time there have been proposals to use different types of data-stores (e.g. NoSQL). At this

point we are not considering adding another data-store for our persistent data. Please read Single data-store.

Single Data-store

We are intentionally requiring that the data for GitLab lives in a single data-store, in this case that means a single database. The term "database" can be taken many different ways. Here is how we are using the following terms:

- database - A database is a named collection of SQL objects ("database objects"). Generally, every database object (tables, functions, etc.) belongs to one and only one database.
- database cluster - A database cluster is a collection of databases that is managed by a single instance of a running database server.
- schema - A database contains one or more named schemas, which in turn contain tables. Schemas also contain other kinds of named objects, including data types, functions, and operators. The same object name can be used in different schemas without conflict; for example, both schema1 and myschema may contain tables named mytable. Unlike databases, schemas are not rigidly separated: a user may access objects in any of the schemas in the database he is connected to, if he has privileges to do so.

Why a single database?

- Our company strategy is built around the advantages of a single application.
 - Sid describes the importance of our single database strategy and the Flywheel with two turbos in this video.
- Our current efforts around database scalability are focusing on database sharding as a solution.

The trouble with microservices

Often, moving to a separate service or microservice is seen as a solution to a scaling or performance problem within the GitLab application. However, moving to a microservice based solution may simply be deferring the problem while creating a more complex architecture. The articles and quotes below discuss real world examples of the struggles with microservices.

- To Microservices and Back Again - Why Segment Went Back to a Monolith

>"If microservices are implemented incorrectly or used as a band-aid without addressing